Computer Programming (1)

Lecture No.9: Functions

Lecturer: Dr. Khaled Mohammed bin Abdl
Department of Information Technology,
College of Applied Science,
Seyion University

Semester 1, 2021

Content

- C++ Modules
- Function syntax
- Function parameters and return types
- Defining functions within programs
- Using and invoking function
- Common errors

Modules

- Blocks of codes with an independent functionality
- Written and tested separately
- Combined to form a complete program
- Modules either control or general
  - Control
    - Called “Main”
      - each program include only one control module
  - General
    - Perform manipulations on data e.g. calculation, print, read and validation

C++ Modules

- All Modules in C++ are called “functions”
  - called methods or procedures in other languages
- Functions can be
  - Programmer-defined
  - Built-in: pow, sqrt, .. etc
- Advantage
  - make programs easier to write, test, debug and maintain

Functions Syntax

Pseudo Code

ModuleName(Parameter List)

1.
2.
3.
4. Exit/Return varaible

```
ReturnType FctName(Parameter List)
{
  action1;
  action2;
    ..
  actionN;
  return value ;
}
```

1 2 3

Function Header

1

2

Function Body

Function Name and Return Type

- Function name follows rules of naming identifiers as variables and constants
- Function return ONLY one value
- Function return types
  - Built-in data types
  - User-defined data types
  - Nothing is returned by using keyword void

```
int f1(Parameter List)
{

  return value ;

}
```

```
void f2(Parameter List)
{

  return;

}
```

```
void f2(Parameter List)
{



}
```

Function Parameters

- Provide the communication links between the main program and its functions
- Parameters declared in the function header within parenthesis
- Either have a single, multiple parameters or have no parameters
- List of a parameters are separated by commas
- Each parameter must be defined as follows

DataType parameterName

```
char f1(int x, int y)
{

  return value ;

}
```

Problem

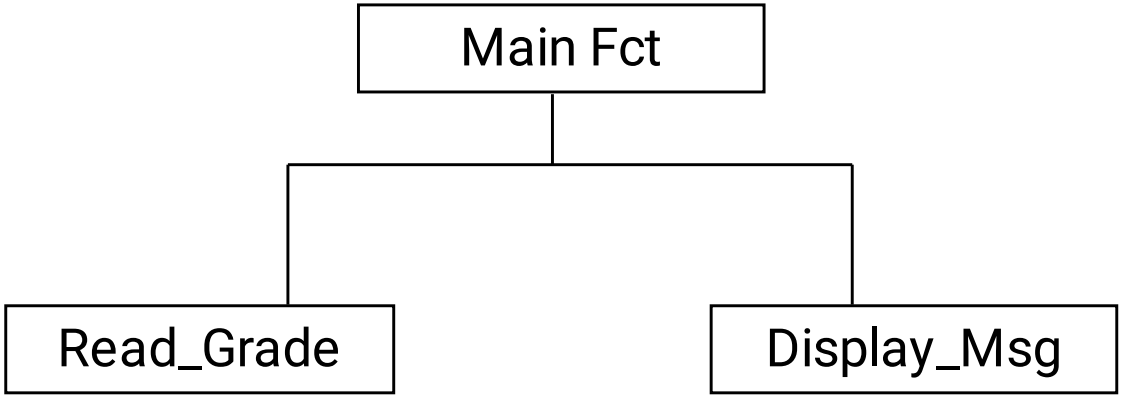
Write a program that asks the user for the exam grade and prints a message indicating if the user has passed the exam or not.

```
#include <iostream>
using namespace std;

int main()
{
    int grade;

    cout<<"Enter your grade:\n";
    cin>>grade;

    if (grade>60)  cout<<"You passed";
    else   cout<<"You failed";

    return 0;
}
```

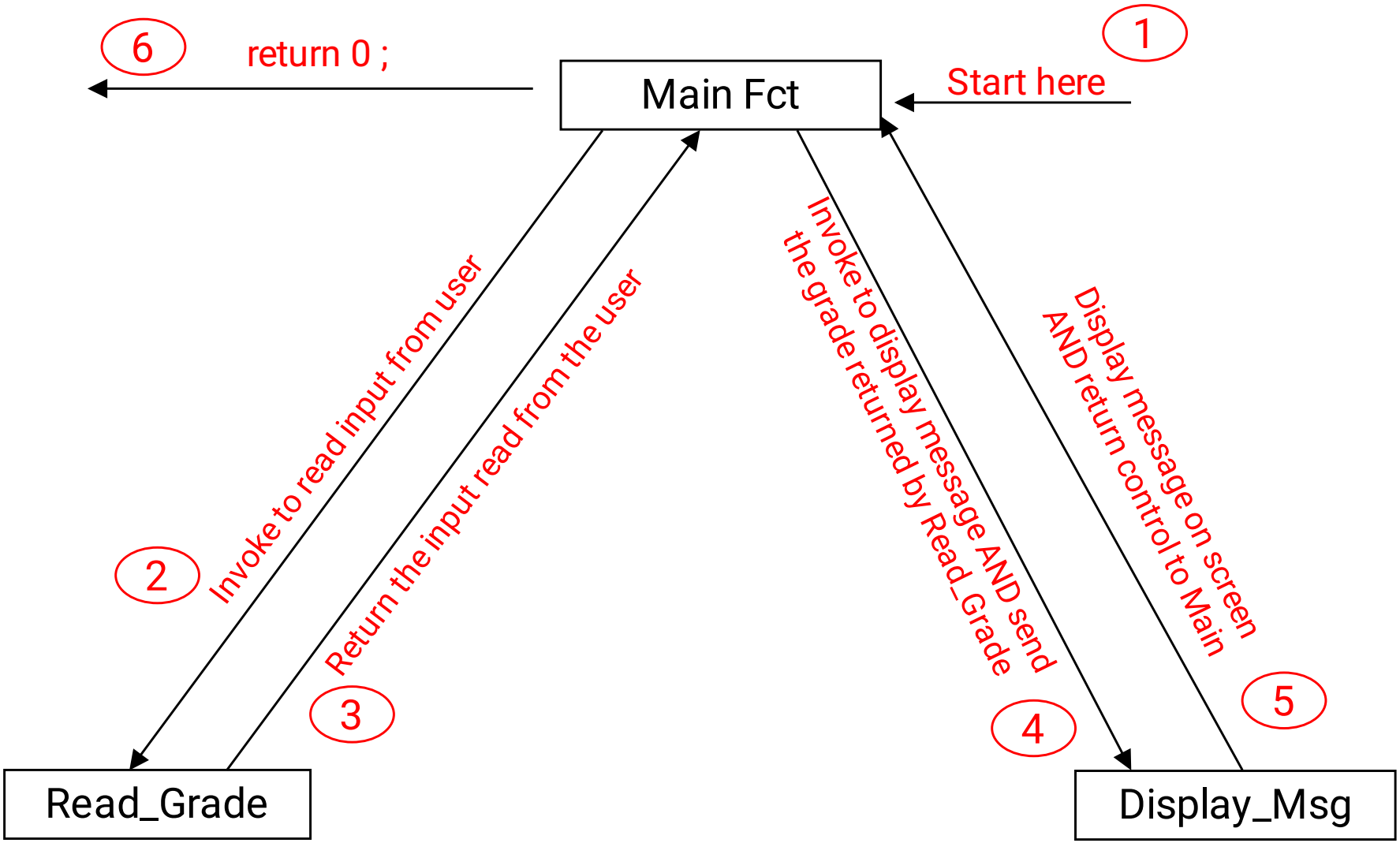
Can you solve the problem using functions ?

1- Selecting Functions

- Divide the program into tasks
  - Read student grade
  - Print appropriate message
- Naming functions
  - Read_Grade
  - Display_Msg
  - main



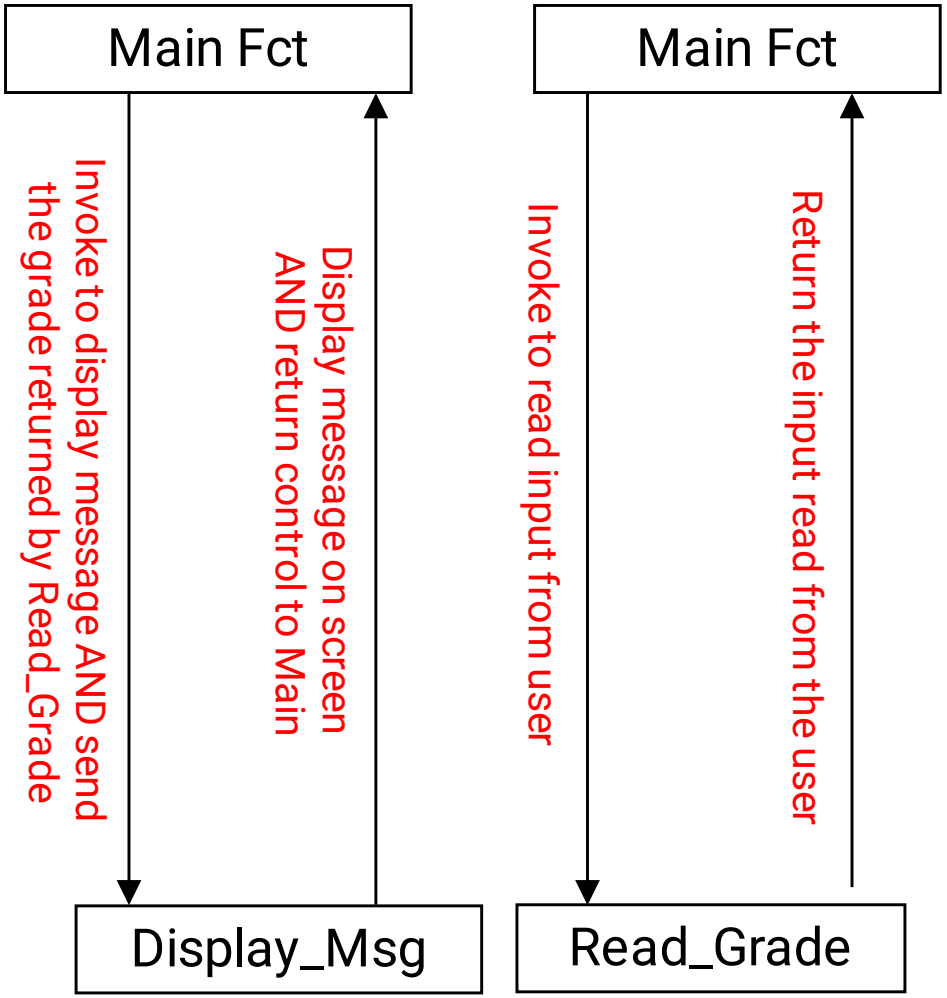
2-Inalyzing Flow of Control



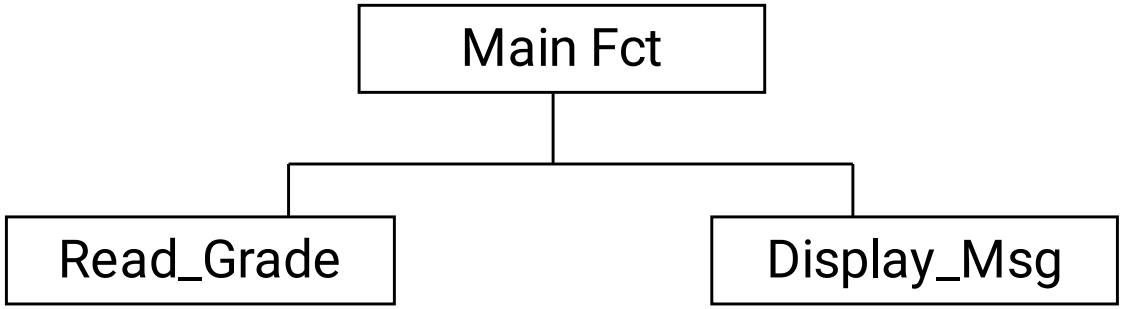
3-Defining Functions

- For each function decide on
  - Possible Parameter List
  - Possible Return Value

```
int  Read_Grade(){

}


void  Display_Message(int g){

}
```



3-Defining Functions (cont.)




```
#include <iostream>
using namespace std;

int main()
{
    int grade;

    cout<<"Enter your grade:\n";
    cin>>grade;

    if (grade>60)  cout<<"You passed";
    else   cout<<"You failed";

    return 0;
}
```

Read_Grade

Display_Msg

3-Defining Functions (cont.)


```
int Read_Grade()
{
    int grade;
    //grade is a local variable
    cout<<"Enter your grade:\n";
    cin>>grade;

    return grade;
}
```

grade scope

```
void Display_Msg(int g)
{
    if (g>60)  cout<<"You passed";
    else   cout<<"You failed";
}
```

g scope

4-Using Functions


```
#include <iostream>
using namespace std;
// ====== You Must Define your Modules before Main ======
int Read_Grade()
{
    int grade;
    //grade is a local variable
    cout<<"Enter your grade:\n";
    cin>>grade;

    return grade;
}
void Display_Msg(int g)
{
    if (g>60)  cout<<"You passed";
    else   cout<<"You failed";
}
// ====================== Main Fct ======================
int main()
{
    // Invoke read module and store returned input
    // Invoke display module and pass the grade
    return 0;
}
```

4-Using Functions (cont.)

- In our example, all invokations occur in Main function
- Invoking C++ functions syntax
  - Write function name followed by ()
  - Within ( ) passing values (arguments) to parameters
    - Functions could have
      - No parameters → **FunctionName()**
      - One or more parameters → **FunctionName(v1,v2)**
  - Capturing returned values
    - Functions return type could be
      - **void** → **FunctionName() OR FunctionName(v1,v2)**
      - **Non-void** → must use returned value in expression

4-Using Functions (cont.)

- Capturing returned values
  - **Non-void** ➔ must use returned value in expression

- Function **Read_grade()** returns an int

  - **int n = Read_Grade();**

  - **if (Read_Grade() == 3) …… … …**

  - **cout << Read_Grade();**

4-Using Functions (cont.)

- Each value passed in a function call is referred to as an Argument
- Arguments (Passed in a Function call) and parameters (defined in a function header) **MUST** match in
  - Number
  - Order
  - Data types

Parameter

```
void Display_Msg(int g)
{
   if (g > 60)
     cout<<"You passed";
   else
     cout<<"You failed";
}
```

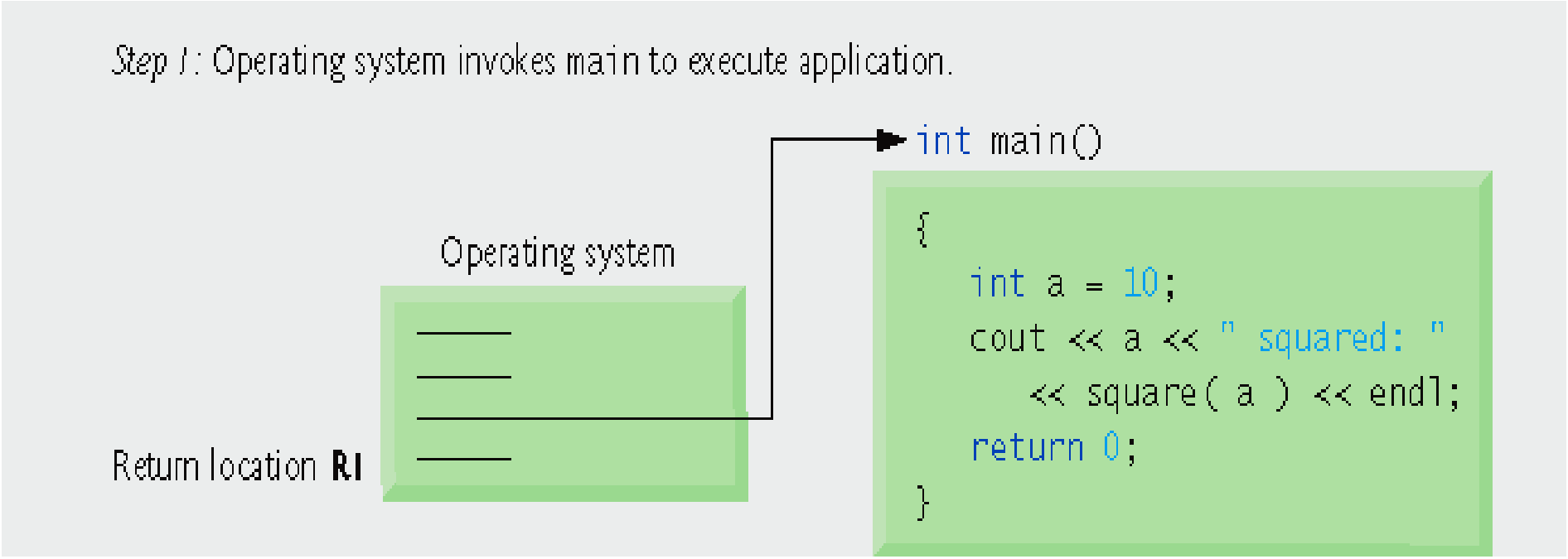
```
int main()
{

     Display_Msg(87);

     return 0;
}
```

Argument

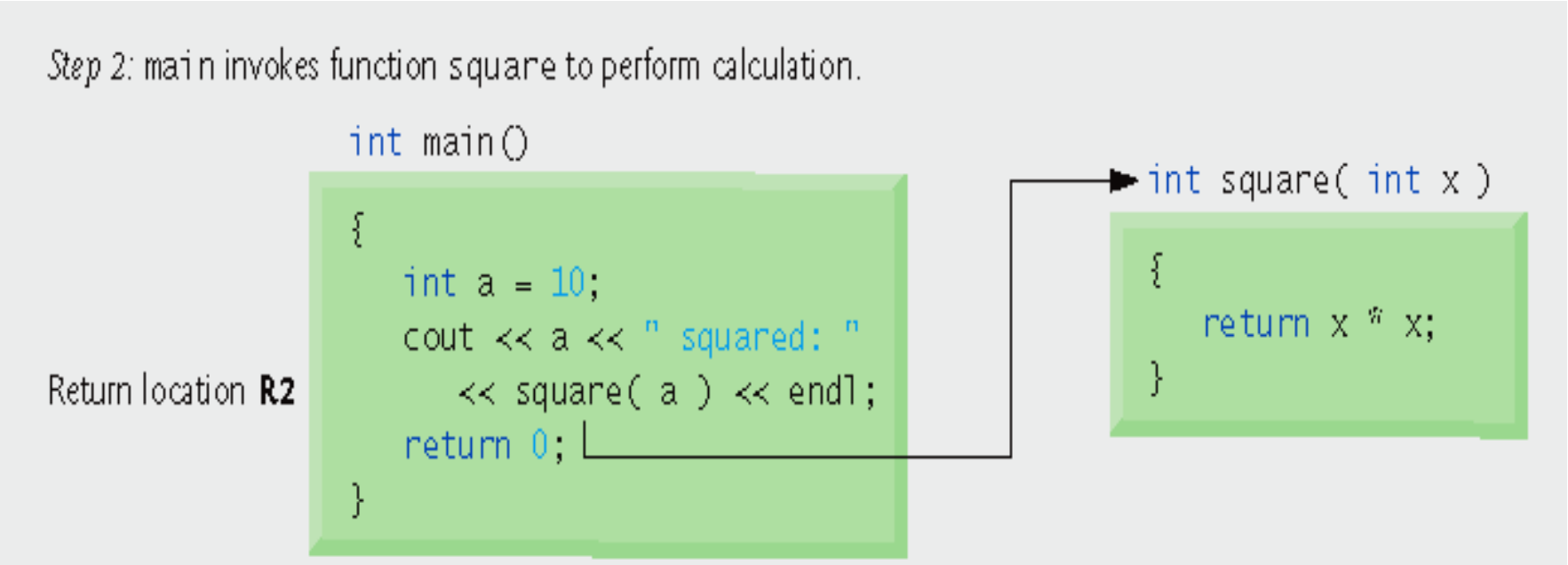
Final Solution


```
#include <iostream>
using namespace std;
// ====== You Must Define your Modules before Main ======
int Read_Grade()
{
    int grade;
    //grade is a local variable
    cout<<"Enter your grade:\n";
    cin>>grade;

    return grade;
}
void Display_Msg(int g)
{
    if (g>60)  cout<<"You passed";
    else   cout<<“You failed";
}
// ====================== Main Fct ======================
int main()
{
    int n = Read_Grade(); // Invoke read function
    Display_Msg(n);      // Invoke display function
    return 0;
}
```

How Function Call Works (1)



How Function Call Works (2)



How Function Call Works (3)

Step 3: square returns its result to main.

```
int main()
{
   int a = 10;
   cout << a << " squared: "
      << square( a ) << endl;
   return 0;
}
```

Return location **R2**

```
int square( int x )
{
   return x * x;
}
```

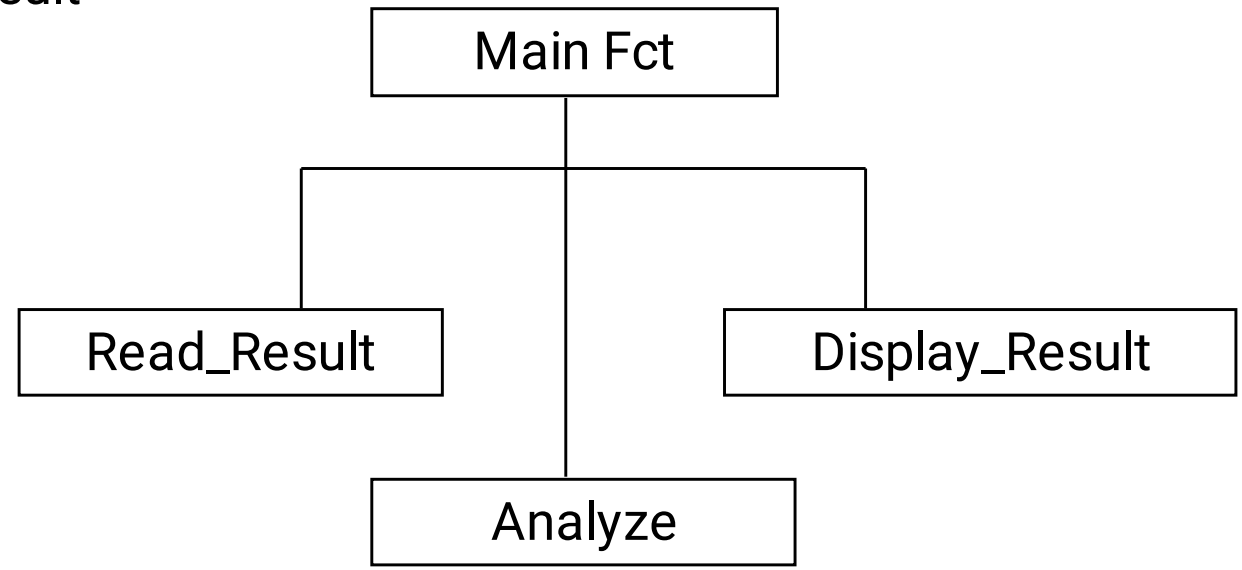
Problem # 2

A college has a list of test results (1 = pass, 2 = fail) for 10 students. Write a program that analyzes the results. If more than 8 students pass, print "Raise Tuition“.

We want to solve this problem using Modules (Functions)

Selecting Modules

- Divide the program into tasks
  - Reading student result
  - Analyzing student result
  - Displaying results
- Naming functions
  - Read_Result
  - Analyze
  - Display_Result
  - Main



```cpp
// Fig. 2.11: fig02_11.cpp
// Analysis of examination results.
#include <iostream>

using std::cout;
using std::cin;
using std::endl;

// function main begins program execution
int main()
{
   // initialize variables in declarations
   int passes = 0;             // number of passes
   int failures = 0;           // number of failures
   int studentCounter = 1;     // student counter
   int result;                 // one exam result

   // process 10 students using counter-controlled loop
   while ( studentCounter <= 10 ) {

      // prompt user for input and obtain value from user
      cout << "Enter result (1 = pass, 2 = fail): ";
      cin >> result;

```

Read_Result

```
25      // if result 1, increment passes; if/else nested in while
26      if ( result == 1 )          // if/else nested in while
27         passes = passes + 1;
28
29      else  // if result not 1, increment failures
30         failures = failures + 1;
31
32      // increment studentCounter so loop eventually terminates
33      studentCounter = studentCounter + 1;
34
35   } // end while
36
37   // termination phase; display number of passes and failures
38   cout << "Passed " << passes << endl;
39   cout << "Failed " << failures << endl;
40
41   // if more than eight students passed, print "raise tuition"
42   if ( passes > 8 )
43      cout << "Raise tuition " << endl;
44
45   return 0;    // successful termination
46
47 } // end function main
```

Analyze

Display_Result

Using Functions


```
#include <iostream>
using namespace std;

// ====================== Main Fct ======================
int main()
{
    int passes=0, failures=0, student_counter=1;
    int result;

    // process 10 students
    while (student_counter <= 10)
    {
     // invoke Read_Result function
     // invoke Analyze function

     student_counter++;
    }

    // invoke Display_Result function

    return 0;
}
```

```
int  Read_Result(){

     int res;
     cout<<"Enter Result ( 1=Pass, 2=Fail): ";
     cin>> res;
     return res;

}


int main()
 {
    int passes=0, failures=0, student_counter=1;
    int result;

    // process 10 students
    while (student_counter <= 10)
    {
        result = Read_Result(); // invoke Read_Result function
      // invoke Analyze function

      student_counter++;
    }

    // invoke Display_Result function

    return 0;
 }
```

Take out to the Global Scope

```
____? Analyze(_______){        int result

                                  ?
if (result==1)  passes++;
  else failures++;
                        ?
  }
```

scope

ONLY one value can return !!!!!!

```
int main()
 {
   int passes=0, failures=0, student_counter=1;
   int result;

   // process 10 students
   while (student_counter <= 10)
   {
      result = Read_Result(); // invoke Read_Result function
      Analyze(result);       // invoke Analyze function

     student_counter++;
   }

   // invoke Display_Result function

   return 0;
```

scope

```
int passes=0, failures=0;

void Analyze(———){        int result

if (result==1)  passes++;
  else failures++;

}
```

scope

```
int main()
 {
   int student_counter=1;
   int result;

   // process 10 students
   while (student_counter <= 10)
   {
      result = Read_Result(); // invoke Read_Result function
      Analyze(result);       // invoke Analyze function

     student_counter++;
   }

   // invoke Display_Result function

   return 0;
```

scope

```
int passes=0, failures=0;

void Display_Result(————){

cout<<"Passed " << passes << endl;
 cout<<"Failures " << failures << endl;

 if ( passes > 8 )
  cout<<"Raise Tution" << endl;
  }

 int main()
  {
     int student_counter=1;
     int result;

     // process 10 students
     while (student_counter <= 10)
     {
         result = Read_Result(); // invoke Read_Result function
         Analyze(result);       // invoke Analyze function

       student_counter++;
     }

     Display_Result();// invoke Display_Result function

     return 0;
  }
```

Make it empty

Using Library Functions

- Need to know the points below to write invokation of built-in functions correctly
  - Name of the function
  - Number , type and order of parameters
  - Type of return value if it has
  - Header file

- Examples

Header File	Function call	Example	Value
<cmath>	pow(x,y)	pow(2,3)	8
<iomanip>	setprecision(n)	setprecision(2)<< 1.2345	1.23

Useful Standard Library Function

- Standard library function **std::pow( )**
  - Calculates an exponent
  - Function takes two arguments of type **double** and returns a **double** value.
  - Requires header file <cmath>
  - Example :

```
#include <cmath>
using std::pow;

//in main function
double x,y,result;
x=2;   y=2;
result = pow(x,y);
```

- Calculates the value of x raised to the yth power

Function	Description	Example
ceil(x)	rounds *x* to the smallest integer not less than *x*	ceil(9.2) is 10.0 ceil(-9.8) is -9.0
cos(x)	trigonometric cosine of *x* (*x* in radians)	cos(0.0) is 1.0
exp(x)	exponential function e^x	exp(1.0) is 2.71828 exp(2.0) is 7.38906
fabs(x)	absolute value of *x*	fabs(5.1) is 5.1 fabs(0.0) is 0.0 fabs(-8.76) is 8.76
floor(x)	rounds *x* to the largest integer not greater than *x*	floor(9.2) is 9.0 floor(-9.8) is -10.0
fmod(x, y)	remainder of *x/y* as a floating-point number	fmod(2.6, 1.2) is 0.2
log(x)	natural logarithm of *x* (base *e*)	log(2.718282) is 1.0 log(7.389056) is 2.0
log10(x)	logarithm of *x* (base 10)	log10(10.0) is 1.0 log10(100.0) is 2.0
pow(x, y)	*x* raised to power *y* (x^y)	pow(2, 7) is 128 pow(9, .5) is 3
sin(x)	trigonometric sine of *x* (*x* in radians)	sin(0.0) is 0
sqrt(x)	square root of *x* (where *x* is a nonnegative value)	sqrt(9.0) is 3.0
tan(x)	trigonometric tangent of *x* (*x* in radians)	tan(0.0) is 0

Math library functions

Common Errors

- Compilation errors
  - Declaring function has parameters of the same type as char a,b instead of char a, char b
  - Mismatching function header with function call in the number, type and order of parameters and arguments
  - Invoking function before it is defined, e.g. when f1 invokes f2, f2 must be defined before f1
  - Attempting to use a function that has no return value in an expression
- Logic errors
  - Undesired implicit typecasting of arguments when they assigned to parameters

Exercise - 1

- Write a function that test a received 2 integer numbers then return the sum then print the result

End, Q & A